B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 9

Specialized Operating Systems: Design and Performance Overview

Conceptual, Analytical, Scenario-Based and HOTS Question Bank

Syllabus Coverage

RTOS: kernel design, scheduling, priority inversion, and performance metrics. Embedded OS: introduction, kernel architecture, task management and scheduling, memory allocation, inter-task communication, and performance metrics. Mobile OS: constraints and design goals, power-aware scheduling, memory management, security and sandboxing models, and performance metrics.

A. Specialized Operating Systems: Design Goals and Trade-offs

1. Why do specialized operating systems often require different design priorities from general-purpose desktop or server operating systems?
2. Compare real-time, embedded, and mobile operating systems in terms of their primary constraints and optimization goals.
3. Explain why average performance alone may be an inadequate metric for evaluating a specialized operating system.
4. How can reliability, predictability, energy efficiency, security, and resource utilization conflict with one another?
5. An operating system must run on hardware with limited CPU capacity and memory while meeting strict response requirements. Which design decisions should be prioritized, and why?
6. Explain how workload characteristics and deployment environment should influence kernel architecture and scheduling policy.

B. Real-Time Operating Systems (RTOS): Kernel Design and Scheduling

7. Define a real-time operating system and distinguish hard, firm, and soft real-time requirements.
8. Explain why determinism and bounded response time are central RTOS design objectives.
9. Compare a monolithic kernel and a microkernel approach for an RTOS, considering latency, fault isolation, and predictability.
10. How can interrupt handling, system-call paths, and context-switch overhead affect real-time responsiveness?
11. Differentiate task execution time, response time, release time, deadline, and period in real-time task models.
12. Compare preemptive and non-preemptive scheduling for real-time workloads.
13. Explain the role of fixed-priority scheduling in an RTOS and identify situations where it is appropriate.
14. How does deadline-based scheduling differ conceptually from fixed-priority scheduling?
15. Why is schedulability analysis important before deploying a real-time application?
16. How can excessive kernel services, dynamic memory allocation, or unbounded operations undermine real-time guarantees?
17. An industrial controller occasionally misses a deadline even though average CPU utilization is low. Analyze possible causes beyond average utilization.
18. How should an RTOS balance low interrupt latency with the need to protect critical sections and maintain system consistency?

C. Priority Inversion and Real-Time Synchronization

19. Define priority inversion and explain how it can occur when high- and low-priority tasks share a resource.
20. Why is priority inversion especially dangerous in systems with deadlines?
21. Describe a scenario in which a medium-priority task indirectly delays a high-priority task through resource contention.
22. Explain priority inheritance and how it limits the effects of priority inversion.
23. Explain priority-ceiling protocols at a conceptual level and identify the kinds of problems they are designed to prevent.
24. Compare priority inheritance and priority ceiling in terms of their general operating principles and trade-offs.
25. Why does priority inheritance mitigate priority inversion but not automatically guarantee that every deadline will be met?
26. How can deadlock and priority inversion interact in a real-time system?
27. Suggest a test strategy for detecting priority inversion and verifying that a synchronization protocol behaves as intended.

D. RTOS Performance Metrics and Evaluation

28. Which performance metrics are most useful for evaluating an RTOS, and why should the metric set depend on the application?
29. Distinguish interrupt latency, dispatch latency, task response time, jitter, and deadline-miss ratio.
30. Why are worst-case execution time and worst-case response time often more important than average values in hard real-time systems?
31. How can timing jitter affect control systems, audio/video pipelines, or communication systems?
32. Explain how to interpret deadline-miss frequency alongside the severity and duration of missed deadlines.
33. Design a conceptual benchmark for an RTOS that measures predictability under high interrupt load and resource contention.

E. Embedded Operating Systems: Architecture and Task Management

34. Define an embedded operating system and explain how its design differs from a general-purpose operating system.
35. Identify the hardware and deployment constraints that commonly shape embedded OS design.
36. Compare bare-metal programming with using an embedded OS. What capabilities are gained and what overheads may be introduced?
37. Explain how kernel architecture affects modularity, memory footprint, fault isolation, and timing predictability in an embedded device.
38. How do tasks, threads, and event-driven execution models support embedded application design?
39. Compare periodic, sporadic, and aperiodic tasks in an embedded system.
40. Why is task prioritization important when an embedded device performs sensing, control, communication, and logging concurrently?
41. How can task-stack sizing and context-switch frequency affect the memory footprint and performance of an embedded OS?
42. An embedded device occasionally fails under peak workload but works reliably during normal operation. Which task-management and resource-contention issues would you investigate?

F. Embedded OS: Memory Allocation and Inter-Task Communication

43. Why can dynamic memory allocation be problematic in memory-constrained or real-time embedded systems?
44. Compare static allocation, dynamic allocation, fixed-size memory pools, and object pools for embedded workloads.
45. Explain internal and external fragmentation and discuss how each can affect an embedded device over long operating periods.
46. Why might a fixed-size memory pool improve predictability while reducing allocation flexibility?

47. Compare queues, shared memory, message passing, semaphores, and event flags as inter-task communication or coordination mechanisms.
48. How should a designer choose between copying messages and sharing a buffer between embedded tasks?
49. What synchronization and ownership problems can arise when multiple tasks access a shared buffer?
50. Explain how buffer overflow, stack overflow, and memory leaks can affect embedded-system reliability.
51. An embedded sensor periodically produces measurements while a communication task transmits them. Propose a communication and buffering design that handles differences in task rates.
52. How can memory protection and defensive programming improve reliability in small embedded systems with limited resources?

G. Embedded OS: Performance Metrics and Validation

53. Which metrics can be used to evaluate an embedded OS beyond raw execution speed?
54. Explain how memory footprint, energy consumption, interrupt latency, task response time, and communication delay influence embedded-system quality.
55. Why should performance tests include peak load, long-duration operation, and resource-exhaustion scenarios?
56. How can instrumentation and tracing help diagnose timing or scheduling problems in an embedded OS?
57. Design a conceptual evaluation plan for an embedded controller that must meet response-time requirements while operating with limited memory and power.

H. Mobile Operating Systems: Constraints and Design Goals

58. What distinguishes a mobile operating system from a desktop operating system in terms of hardware, usage patterns, and environmental constraints?
59. Explain how battery capacity, thermal limits, wireless connectivity, and variable workloads influence mobile OS design.
60. Why must a mobile OS balance responsiveness with energy efficiency?
61. How can foreground, background, and user-visible activity influence resource-management policy?
62. Explain the tension between application responsiveness, background execution, and battery preservation.
63. Why are thermal throttling and sustained performance relevant to mobile OS evaluation?

I. Mobile OS: Power-Aware Scheduling and Memory Management

64. Explain the purpose of power-aware scheduling and identify the information an OS might use when making scheduling decisions.
65. How can CPU frequency scaling and idle-state management reduce energy consumption? What performance trade-offs may result?
66. Why can frequently waking the processor or radio increase energy consumption even when individual operations are short?
67. How can a mobile OS decide which background tasks should be delayed, limited, or allowed to execute immediately?
68. Explain how memory pressure can lead a mobile OS to reclaim caches, compress memory, terminate background processes, or use other recovery strategies.
69. Why might a mobile OS terminate a background application even when the user has not explicitly closed it?
70. Compare the goals of memory management on a mobile device with those on a memory-rich server.
71. How can aggressive memory reclamation improve foreground responsiveness but harm user experience?
72. An application experiences repeated background termination and slow restoration. Analyze possible interactions between memory pressure, process lifecycle policy, and application state management.

J. Mobile OS: Security, Sandboxing, and Performance Metrics

73. Explain the purpose of application sandboxing in a mobile operating system.
74. How do process isolation, permission controls, and least-privilege design reduce the impact of a compromised application?
75. Compare permission-based access control with isolation mechanisms such as per-application identities and restricted system interfaces.
76. Why must a mobile OS manage sensitive resources such as location, camera, microphone, contacts, and persistent storage carefully?
77. How can background-execution restrictions contribute to both security and energy efficiency?
78. Discuss the trade-off between strong application isolation and convenient inter-application communication.
79. Which metrics should be used to evaluate a mobile OS across responsiveness, energy consumption, memory pressure, security, and thermal behavior?
80. Why should mobile performance testing include both short interactive tasks and long-duration workloads?
81. Design a conceptual test plan for a mobile OS that evaluates battery use, application launch time, memory pressure, and sandbox enforcement.

K. Integrative and Higher-Order Thinking (HOTS)

82. A medical monitoring device must process sensor data predictably, communicate alerts, and conserve battery power. Compare the design priorities of an RTOS, embedded OS, and mobile OS for this scenario.
83. A high-priority real-time task misses its deadline because a low-priority task holds a shared lock. Explain the failure and recommend a synchronization protocol and validation approach.
84. An embedded product has a small memory budget and must operate continuously for months. Recommend a memory-allocation strategy and explain how you would evaluate fragmentation and failure risk.
85. A mobile application is responsive while in the foreground but becomes delayed after the screen is turned off. Analyze possible causes involving power-aware scheduling and background-execution policy.
86. A mobile device becomes hot during sustained workloads even though short benchmark tests look excellent. Explain why the evaluation methodology is inadequate and propose better metrics.
87. Critically evaluate the claim that a microkernel is always the best architecture for a safety-critical or embedded operating system.
88. Develop a comparison framework for RTOS, embedded OS, and mobile OS that uses measurable metrics rather than vague labels such as fast, lightweight, or secure.
89. An embedded gateway combines strict control deadlines, intermittent network connectivity, and untrusted third-party applications. Identify the key architecture, scheduling, memory, communication, and security decisions.